

КУРСОВАЯ РАБОТА

по дисциплине «Объектно-ориентированное программирование»

МОДЕЛИРОВАНИЕ 2–3-ДЕРЕВА

Разработка и исследование сбалансированной структуры данных
на языке C++

Выполнил(а): студент(ка) группы КМБО-11-24
Пименов Евгений Юрьевич

Проверил(а): ДОЛЖНОСТЬ, ФИО

Оценка: _____
Дата: _____

[ГОРОД] — 2026

ОГЛАВЛЕНИЕ

Введение.....	3
Глава 1. Теоретические основы 2–3-дерева.....	4
1.1. Назначение сбалансированных деревьев поиска.....	4
1.2. Свойства и представление 2–3-дерева.....	4
1.3. Операции и оценка высоты.....	5
1.4. Удаление и упорядоченные запросы.....	6
1.5. Сравнение с альтернативными структурами.....	7
Глава 2. Практическая реализация на C++17.....	8
2.1. Архитектура и система классов.....	8
2.2. Представление узла и управление памятью.....	9
2.3. Поиск и добавление элементов.....	10
2.4. Удаление и восстановление инвариантов.....	11
2.5. Упорядоченные запросы и объединение.....	12
2.6. Поточковый ввод и вывод.....	13
2.7. Пользовательский интерфейс.....	14
2.8. Результаты работы программы.....	15
2.9. Тестирование, оценки и ограничения.....	16
Заключение.....	17
Список использованной литературы.....	18
Приложение А. Сборка и примеры использования.....	19

ВВЕДЕНИЕ

При обработке больших динамически изменяемых множеств часто требуется не только быстро находить элемент по ключу, но и поддерживать строгий порядок элементов. Типичные операции — поиск ближайшего большего либо меньшего значения, перечисление ключей в интервале, добавление новых записей и удаление устаревших. Линейные структуры хорошо подходят для последовательного просмотра, однако затраты на поиск или изменение в общем случае могут оказаться значительными.

Одним из решений являются сбалансированные деревья поиска. Они сохраняют ограниченную высоту независимо от порядка поступления значений. К числу таких структур относится 2–3-дерево: каждый его внутренний узел имеет два или три дочерних узла, а все листья расположены на одинаковой глубине. Благодаря этому операции поиска и изменения требуют логарифмического числа переходов между узлами [1, 2].

Цель курсовой работы — разработать и исследовать программную модель 2–3-дерева, поддерживающую хранение целочисленного множества, поиск, вставку, удаление, объединение, поиск соседних ключей и получение m следующих элементов. Отдельная задача — организовать универсальный потоковый ввод и вывод, пригодный для работы как с консолью, так и с файлами.

Объектом исследования выступают динамические упорядоченные множества. Предмет исследования — алгоритмы сбалансированного хранения элементов и особенности их реализации с использованием классов, наследования, динамической памяти и перегрузки операций в C++17.

В первой главе рассматриваются устройство 2–3-дерева и его место среди альтернативных структур данных. Во второй главе описывается созданное приложение: архитектура классов, назначение методов, алгоритмы балансировки, обработка пользовательского ввода и результаты тестирования. В приложении приведены команды сборки и примеры использования программного интерфейса.

При реализации принята модель В-дерева порядка 3: ключи хранятся как в листьях, так и во внутренних узлах. В некоторых источниках [3] описано другое представление — с ключами в листьях и разделителями во внутренних вершинах. Данные подходы сохраняют одинаковые асимптотические свойства, но требуют разных процедур обработки узлов.

ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ 2–3-ДЕРЕВА

1.1. Назначение сбалансированных деревьев поиска

Двоичное дерево поиска поддерживает инвариант упорядоченности: ключи левого поддерева меньше ключа корня, а правого — больше. Если входные ключи подаются в отсортированном порядке, дерево без балансировки может вырождаться в цепочку, и сложность поиска возрастает до $O(n)$. Высотно-сбалансированные структуры ограничивают длину пути от корня к листу, сохраняя сложность $O(\log n)$ и для неблагоприятных последовательностей вставки [2].

1.2. Свойства и представление 2–3-дерева

2–3-дерево — дерево поиска, в устойчивом состоянии удовлетворяющее следующим требованиям: (1) узел содержит один или два упорядоченных ключа; (2) нелистовой узел с одним ключом имеет двух потомков, с двумя — трёх; (3) все листья находятся на одном уровне; (4) диапазоны ключей дочерних поддеревьев не пересекаются. Повторы ключей в исследуемой реализации не хранятся: структура представляет математическое множество.

Пусть в узле содержится один ключ k . Тогда каждый элемент левого поддерева меньше k , а каждый элемент правого — больше k . Для узла с ключами $k_1 < k_2$ существуют три области: ключи меньше k_1 ; значения строго между k_1 и k_2 ; ключи больше k_2 . Порядок ключей и ограничения на дочерние поддеревья поддерживаются при каждом изменении структуры.

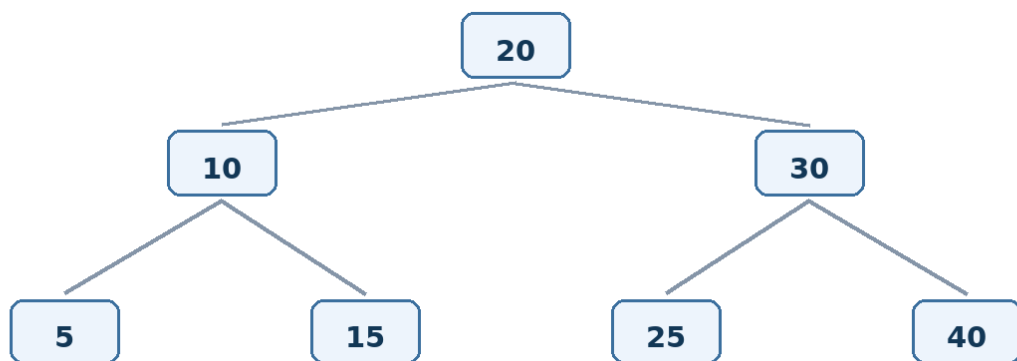


Рисунок 1. 2–3-дерево после последовательной вставки ключей 10, 20, 5, 15, 30, 25, 40

В узлах рисунка ключи представлены как самостоятельные элементы дерева; значения во внутренних узлах не дублируются в листьях.

1.3. ОПЕРАЦИИ И ОЦЕНКА ВЫСОТЫ

Поиск начинается с корня. В каждом узле сравнивают заданное значение с одним или двумя ключами: при совпадении поиск успешен; иначе выбирают соответствующего потомка. Проверка текущего узла требует $O(1)$ сравнений. Число посещённых узлов определяется высотой дерева h , поэтому стоимость операции составляет $O(h)$.

При вставке спускаются к подходящему листу. Если там имеется один ключ, новый ключ размещают в упорядоченной позиции. Если после вставки узел содержит три ключа $[a \mid b \mid c]$, он временно переполнен. Средний ключ b переносится в родительский узел; остаются два узла $[a]$ и $[c]$. Переполнение может подняться выше вплоть до корня. Если разделяется корень, создаётся новый корень, и общая высота увеличивается ровно на один уровень.

```
// Расщепление ребёнка, содержащего три ключа.  
Node& left = *parent.children[index];  
const int middle = left.keys[1];  
auto right = std::make_unique<Node>(left.keys[2]);  
left.keys.resize(1);  
parent.keys.insert(parent.keys.begin() + index, middle);
```

Листинг 1. Фрагмент алгоритма расщепления переполненного узла

Логарифмическую высоту можно вывести из ограничений на степень узлов. При высоте h (число рёбер от корня до листа) минимальное число ключей в непустом дереве не меньше $2^{(h+1)} - 1$, а максимальное — не больше $3^{(h+1)} - 1$. Следовательно, h ограничена величиной $O(\log n)$, где n — число ключей, и поиск, вставка и удаление требуют $O(\log n)$ времени [1, 4].

Процедура вставки в программе носит рекурсивный характер: на спуске определяется нужный лист, на возврате восстанавливаются инварианты. Поскольку на каждом уровне обрабатывается постоянное число ключей, асимптотика сохраняется даже при каскадном разделении вершин.

1.4. УДАЛЕНИЕ И УПОРЯДОЧЕННЫЕ ЗАПРОСЫ

Удаление считается более сложной операцией, чем вставка. Если ключ расположен во внутреннем узле, его заменяют ближайшим меньшим ключом из левого поддерева и удаляют найденное значение из листа. Если лист после удаления остаётся с одним ключом, структура корректна. В противном случае образуется временный дефицит — узел с нулём ключей.

Дефицит устраняют сначала заимствованием у непосредственного левого либо правого соседа, если сосед содержит два ключа. Между вершинами перемещается разделительный ключ родителя, а при необходимости также переносится соответствующее дочернее поддерево. Если оба соседа минимальны, дефицитный узел сливается с соседом с использованием одного ключа родителя. В результате сам родитель может потерять последний ключ: восстановление продолжается вверх. Пустой корень заменяется единственным потомком, что уменьшает высоту дерева на один уровень [2].

Помимо точного поиска, упорядоченное дерево позволяет получить соседние ключи. Следующий ключ (successor) — минимальный элемент множества, строго больший x ; предыдущий (predecessor) — максимальный элемент, строго меньший x . Во время спуска по дереву достаточно сохранять лучший найденный кандидат; стоимость обоих запросов равна $O(\log n)$. Если подходящего ключа нет, в C++ возвращается `std::nullopt`.

Нахождение m следующих элементов можно реализовать m последовательными вызовами `successor`, что даст $O(m \log n)$. Более эффективен обход, который пропускает все поддеревья с ключами, не превосходящими x , и прекращается после получения необходимого числа результатов. Для реализации в этой работе трудоёмкость такого запроса составляет $O(\log n + r)$, где r — фактическое число возвращённых ключей, $r \leq m$.

Операция объединения деревьев интерпретируется как объединение множеств: ключ, имеющийся в обоих деревьях, попадает в результат один раз. В разработанном варианте создаётся копия первого дерева, после чего в неё последовательно вставляются все ключи второго. Такой подход прост и надёжен, хотя не достигает оптимальности специализированных алгоритмов слияния деревьев одинаковой высоты.

1.5. СРАВНЕНИЕ С АЛЬТЕРНАТИВНЫМИ СТРУКТУРАМИ

Основные задачи 2–3-дерева могут решаться и другими структурами данных. Выбор зависит от того, что важнее: скорость одиночного поиска, поддержание порядка, частые изменения, использование внешней памяти либо простота реализации. В таблице 1 приведены асимптотические оценки наиболее распространённых решений [4, 5].

Таблица 1. Сравнение структур хранения упорядоченных множеств

Структура	Поиск	Вставка / удаление	m следующих
2–3-дерево	$O(\log n)$	$O(\log n)$	$O(\log n + m)$
AVL-дерево	$O(\log n)$	$O(\log n)$	$O(\log n + m)$
Красно-чёрное	$O(\log n)$	$O(\log n)$	$O(\log n + m)$
Декартово дерево	$O(\log n)$ в среднем	$O(\log n)$ в среднем	$O(\log n + m)$ в среднем
Хеш-таблица	$O(1)$ в среднем	$O(1)$ в среднем	Нет упорядоченного обхода
Отсортир. массив	$O(\log n)$	$O(n)$	$O(\log n + m)$

AVL-дерево часто даёт небольшую высоту поиска, однако после изменения может потребовать вращений. Красно-чёрное дерево обеспечивает логарифмические оценки и широко применяется для реализации контейнеров `std::set` и `std::map`. Красно-чёрные представления можно рассматривать как бинарную кодировку многоузловых сбалансированных деревьев, что разъясняется в учебных материалах [6].

В-деревья и В+ деревья обобщают 2–3-дерево на более крупные узлы, содержащие много ключей. При размещении индексов на диске такая организация уменьшает число обращений к блокам. Хеш-таблица обычно быстрее для точного поиска, но не поддерживает соседние ключи и упорядоченные диапазоны без дополнительных механизмов. Массив удобен для преимущественно статических данных.

Для данной работы 2–3-дерево выбрано потому, что позволяет продемонстрировать не только упорядоченность и логарифмическую высоту, но и локальные преобразования структуры: разделение, заимствование, слияние и изменение высоты корня. Именно эти механизмы составляют основной предмет исследования.

ГЛАВА 2. ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ НА C++17

2.1. Архитектура и система классов

Программа разделена на интерфейс классов (include/TwoThreeTree.hpp), реализацию алгоритмов (src/TwoThreeTree.cpp), консольную оболочку (src/main.cpp) и автоматические тесты (tests/test.cpp). Для сборки предусмотрены компилятор C++17 и проект CMake. Такое разделение позволяет тестировать алгоритмы независимо от пользовательского интерфейса.



Рисунок 2. Упрощённая схема наследования и композиции классов

AbstractSearchTree задаёт единый полиморфный контракт: `insert`, `erase`, `contains`, `successor`, `predecessor`, `nextElements`, `size` и `printStructure`. Производный класс **TwoThreeTree** реализует эти методы, а также содержит конструкторы, операции копирования и перемещения, объединение деревьев, сортированный обход, очистку и проверку инвариантов.

Вложенная структура **Node** не наследуется от **AbstractSearchTree**: она является деталью реализации и хранит ключи и указатели на детей. Закрытая область `private` защищает внутреннюю структуру от изменения вызывающим кодом. Отдельное размещение интерфейса и реализации снижает связанность модулей.

2.2. ПРЕДСТАВЛЕНИЕ УЗЛА И УПРАВЛЕНИЕ ПАМЯТЬЮ

Дерево хранит два основных поля: `root_` — владеющий указатель на корень, `size_` — число уникальных ключей. Класс `Node` содержит динамические массивы `keys` и `children`. Для управления поддеревьями используется `std::unique_ptr<Node>`. Такой подход обеспечивает автоматическое освобождение памяти при удалении узла и исключает необходимость явно вызывать `delete` при штатной работе программы.

```
private:
    struct Node {
        std::vector<int> keys;
        std::vector<std::unique_ptr<Node>> children;
        Node() = default;
        explicit Node(int value) : keys{value} {}
        bool isLeaf() const noexcept { return children.empty(); }
    };
    std::unique_ptr<Node> root_;
    std::size_t size_ = 0;
```

Листинг 2. Внутренняя модель узла и состояние дерева

В устойчивом состоянии допустимы только один или два ключа. Во время вставки `vector` временно может вместить третий ключ до процедуры `splitChild`. Во время удаления допускается кратковременное отсутствие ключей в проблемном узле до вызова `fixUnderflow`. Это внутренние промежуточные состояния, после завершения публичной операции инварианты должны быть восстановлены.

Конструктор с параметром `std::vector<int>` последовательно вызывает `insert` для входных значений. Копирующий конструктор создаёт новое дерево из ключей исходного — поэтому копия не делит указатели `Node` с оригиналом. Копирующее присваивание использует временный объект, затем перемещает его в назначение. Перемещающие операции передают владение корнем и обнуляют размер источника.

Метод `clear()` вызывает `root_.reset()` и обнуляет `size_`. Виртуальный деструктор базового класса позволяет безопасно уничтожать производный объект через указатель на базовый интерфейс. Для чтения без изменения используются `const`-методы. Методы `empty()` и `size()` имеют спецификатор `noexcept`, поскольку только читают счетчик элементов.

2.3. ПОИСК И ДОБАВЛЕНИЕ ЭЛЕМЕНТОВ

Публичный метод `contains(key)` выполняет итеративный поиск. На каждом уровне `std::lower_bound` находит первый ключ узла, не меньший заданного. Если найдено точное совпадение — возвращается `true`; если совпадения нет и достигнут лист — `false`. Иначе управление передаётся нужному дочернему узлу. Так как узел содержит не больше двух ключей, бинарный поиск по его массиву фактически требует постоянное число сравнений.

```
bool TwoThreeTree::contains(int key) const {
    const Node* node = root_.get();
    while (node) {
        auto it = std::lower_bound(node->keys.begin(),
                                   node->keys.end(), key);
        if (it != node->keys.end() && *it == key) return true;
        if (node->isLeaf()) return false;
        node = node->children[it - node->keys.begin()].get();
    }
    return false;
}
```

Листинг 3. Алгоритм поиска ключа (сокращённый пример)

Метод `insert(key)` предварительно вызывает `contains(key)`: это обеспечивает отсутствие повторяющихся ключей и позволяет вернуть `false`, если элемент уже хранится. Если дерево пусто, создаётся корень с единственным ключом. В остальных случаях `insertRecursive` спускается к листу, вставляет ключ в нужную позицию и на обратном пути проверяет переполнение дочернего узла.

Метод `splitChild(parent, index)` создаёт новый правый узел, переносит в него старший ключ и при наличии детей — два правых поддерева. Средний ключ вставляется в родителя, а исходный узел сохраняет младший ключ. Если переполнился корень, `insert()` создаёт новый корень, выполняет разделение и заменяет `root_`. После успешной операции увеличивается `size_`.

Данная организация не требует указателей на родителей и локализует изменения на единственном пути поиска. Отсутствие дублирующих значений означает, что методы удаления, объединения и сериализации интерпретируют дерево именно как множество, а не мультимножество.

2.4. УДАЛЕНИЕ И ВОССТАНОВЛЕНИЕ ИНВАРИАНТОВ

Удаление реализовано парой `erase(key)` и `eraseRecursive(node, key)`. Рекурсивная процедура определяет положение заданного ключа относительно ключей текущей вершины. Когда элемент обнаружен во внутреннем узле, он заменяется максимальным значением из левого поддерева; реальное удаление затем выполняется в листе. После рекурсивного вызова проверяется, не остался ли соответствующий потомок без ключей.

Метод `fixUnderflow(parent, index)` последовательно рассматривает три возможных способа восстановления: заимствование у левого соседа с двумя ключами, заимствование у правого соседа с двумя ключами и слияние с ближайшим соседом при невозможности заимствования. Для внутренних узлов одновременно перераспределяются и указатели на дочерние деревья. При слиянии один разделительный ключ извлекается из родителя, вследствие чего дефицит может распространиться вверх.

```
bool TwoThreeTree::erase(int key) {
    if (!root_ || !eraseRecursive(*root_, key)) return false;
    --size_;
    if (root_>keys.empty()) {
        if (root_>isLeaf()) root_.reset();
        else root_ = std::move(root_>children.front());
    }
    return true;
}
```

Листинг 4. Завершение удаления с возможным понижением корня

Особое внимание уделено состоянию корня: если он полностью теряет ключи и имеет потомка, единственный потомок становится новым корнем. Благодаря такому преобразованию высота дерева уменьшается только целиком, и равная глубина листьев сохраняется.

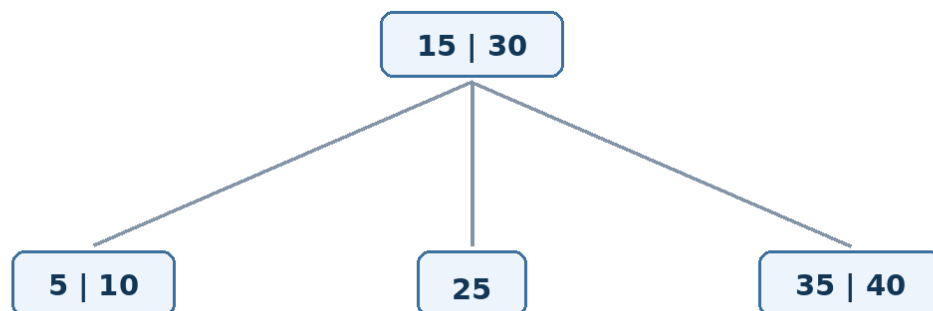


Рисунок 3. Дерево после вставки 35 и удаления 20 из исходного примера

2.5. УПОРЯДОЧЕННЫЕ ЗАПРОСЫ И ОБЪЕДИНЕНИЕ

Методы `successor` и `predecessor` возвращают `std::optional<int>`. Это позволяет различать корректное значение ключа и отсутствие ответа, не вводя специальные числа-маркеры. Поиск производится итеративно, причём на каждом уровне запоминается потенциально лучший ключ, удовлетворяющий строгому неравенству.

Метод `nextElements(key, m)` возвращает не более `m` ближайших ключей, строго больших `key`, в порядке возрастания. Он передаёт выполнение `nextRecursive`, которая отсекает поддеревья, не содержащие подходящих значений, и завершает обход сразу после наполнения результирующего массива. Для `m = 0` возвращается пустой вектор. Если в множестве меньше `m` подходящих ключей, выдаются все доступные значения.

```
std::vector<int> TwoThreeTree::nextElements(  
    int key, std::size_t m) const {  
    std::vector<int> result;  
    result.reserve(std::min(m, size_));  
    nextRecursive(root_.get(), key, m, result);  
    return result;  
}
```

Листинг 5. Открытый метод получения `m` следующих ключей

Метод `toVector()` выполняет симметричный обход дерева и формирует отсортированный массив ключей. Он используется не только для демонстрации элементов, но также в копировании, сравнении результатов в тестах и форматировании потокового вывода.

Объединение представлено методом `merged(other)`, возвращающим новый объект `TwoThreeTree`. Сначала создаётся глубокая копия текущего дерева, затем элементы второго дерева перечисляются в возрастающем порядке и добавляются через `insert`. Дубликаты автоматически отбрасываются. Оба исходных дерева после выполнения операции остаются неизменными; это проверяется в тестовом наборе.

Стоимость объединения двух деревьев мощностей `n` и `k` в данной реализации имеет верхнюю оценку $O((n+k) \log(n+k))$, дополнительная память — $O(n+k)$. Специализированное слияние отсортированных последовательностей и последующее построение дерева могло бы ускорить операцию, но потребовало бы отдельного алгоритма линейного построения В-дерева из упорядоченных ключей.

2.6. ПОТОКОВЫЙ ВВОД И ВЫВОД

Для работы с консолью и файловой системой перегружены операторы `<<` и `>>`. Оба принимают базовые типы `std::ostream` и `std::istream`, поэтому одинаковый код можно использовать с `std::cout`, `std::cin`, `std::ofstream`, `std::ifstream` и `std::stringstream`. Методы не зависят от конкретного источника либо получателя данных.

Принят текстовый формат «N K1 K2 ... KN», где N — количество перечисленных ключей, а затем N целых значений. При записи ключи всегда сортируются и повторов нет. При чтении порядок произволен: повторяющиеся значения не добавляются, поэтому размер полученного дерева может быть меньше N. Структурная форма дерева не сериализуется — в файле сохраняется содержимое множества.

```
std::ostream& operator<<(std::ostream& out,
                        const TwoThreeTree& tree) {
    out << tree.size_;
    for (int value : tree.toVector()) out << ' ' << value;
    return out;
}
std::istream& operator>>(std::istream& in,
                        TwoThreeTree& tree) {
    long long count;
    if (!(in >> count)) return in;
    if (count < 0) { in.setstate(std::ios::failbit); return in; }
    TwoThreeTree parsed;
    for (long long i = 0; i < count; ++i) {
        int key;
        if (!(in >> key)) return in;
        parsed.insert(key);
    }
    tree = std::move(parsed);
    return in;
}
```

Листинг 6. Перегруженные операторы потокового ввода и вывода

Функции возвращают ссылки на потоки, чтобы поддерживать привычное цепочечное использование и проверку состояния потока: `in >> a >> b`, `out << a << b`. Важно и создание временного дерева `parsed`: исходный объект изменяется только после полного успешного чтения, поэтому неполный файл не повреждает хранимые данные. Отрицательное значение N приводит к установке бита `failbit`.

2.7. ПОЛЬЗОВАТЕЛЬСКИЙ ИНТЕРФЕЙС

Файл `main.cpp` реализует интерактивное текстовое меню. После запуска создаётся пустой объект `TwoThreeTree`, затем в цикле считывается номер команды. Команды сгруппированы по назначению: создание и отображение, изменение множества, поиск и соседние ключи, выборка диапазона, объединение, файловые операции и проверка состояния структуры.

Таблица 2. Команды консольного приложения

Команды	Назначение
1; 2	Создание дерева по последовательности; просмотр ключей и уровней.
3; 4; 5	Добавление, удаление и проверка наличия ключа.
6; 7	Ближайшие соседние ключи; поиск m следующих значений.
8	Объединение с деревом, введённым пользователем.
9; 10	Чтение из текстового файла; сохранение в файл.
11; 0	Проверка инвариантов структуры; завершение программы.

При командах 1 и 8 пользователь вводит число N и затем N целых ключей. Для команд 3–6 достаточно одного целого ключа; для команды 7 вводятся `key` и `m`. При файловых операциях вводится путь к файлу одним словом без пробелов. Функция `printStructure` выводит ключи на каждом уровне дерева, нумеруя уровни начиная с корня.

Предусмотрена часть механизмов защиты от некорректных данных. Дубликаты игнорируются; попытка повторно добавить ключ возвращает сообщение «Уже существует», а удаление отсутствующего значения — «Не найден». При чтении дерева из файла неверный N , недостаточное количество ключей и отсутствие доступного файла приводят к сообщению об ошибке, при этом ранее построенное дерево сохраняется. Ошибка формата при создании дерева через консоль завершает работу приложения.

Ограничение интерфейса: если пользователь вводит текст вместо ожидаемого числа в главном потоке `std::cin`, работа программы завершается, а не повторяет запрос. Значение `m` читается в беззнаковый тип `std::size_t` без явной проверки отрицательного ввода; при развитии приложения следует предусмотреть отдельную проверку границ и безопасное повторное приглашение.

2.8. РЕЗУЛЬТАТЫ РАБОТЫ ПРОГРАММЫ

Для проверки базовых операций проведён воспроизводимый запуск консольного приложения. В первом сценарии дерево создаётся из последовательности «7 10 20 5 15 30 25 40». После выполнения команды 2 программа выводит сериализацию множества и структуру дерева по уровням. Результат соответствует рисунку 1.

```
Создано. Размер: 7
Сериализация: 7 5 10 15 20 25 30 40
Уровень 0: [20]
Уровень 1: [10] [30]
Уровень 2: [5] [15] [25] [40]
Ключ: 15 -> Найден
Ключ: 15 -> Следующий: 20; предыдущий: 10
Ключ и m: 12 3 -> Найдено 3: 15 20 25
```

Листинг 7. Фрагмент вывода программы: создание и упорядоченные запросы

Во втором сценарии выполняется insert(35), затем erase(20). После удаления внутреннего ключа структура перестраивается: новый корень содержит значения [15|30], а листовой уровень состоит из узлов [5|10], [25], [35|40]. Несмотря на изменения структуры, обход остаётся строго возрастающим и число ключей равно семи.

```
Ключ: 35 -> Добавлен
Ключ: 20 -> Удалён
Сериализация: 7 5 10 15 25 30 35 40
Уровень 0: [15|30]
Уровень 1: [5|10] [25] [35|40]
Инварианты соблюдены
```

Листинг 8. Фрагмент вывода программы после изменения множества

В третьем сценарии создаётся второе дерево с ключами 0, 15, 45, 35. После объединения количество уникальных значений становится равным девяти. Вывод сохраняет порядок и не содержит дубликатов ключей 15 и 35.

```
Объединено. Размер: 9
Сериализация: 9 0 5 10 15 25 30 35 40 45
Уровень 0: [25]
Уровень 1: [10] [35]
Уровень 2: [0|5] [15] [30] [40|45]
```

Листинг 9. Фрагмент вывода программы после объединения деревьев

2.9. ТЕСТИРОВАНИЕ, ОЦЕНКИ И ОГРАНИЧЕНИЯ

Проверка выполнена двумя способами: интерактивно, с контролем выводимых значений, и автоматически с использованием `assert`. Автоматические тесты проверяют пустое дерево, последовательную вставку 1000 возрастающих ключей, удаление их в обратном порядке и 25 000 псевдослучайных операций вставки/удаления при фиксированном начальном состоянии генератора. Эталонной структурой для сравнения служит `std::set<int>`.

После каждого изменения вызывается `validate()`, которая проверяет допустимое количество ключей в узлах, порядок ключей и диапазоны поддеревьев, правильное число детей, равенство глубин всех листьев и согласованность счётчика `size_`. Дополнительно тестируются `successor`, `predecessor`, `nextElements`, операции копирования/перемещения, объединение и потоковая сериализация.

Таблица 3. Результаты функциональных проверок

Проверяемый сценарий	Ожидаемый результат	Факт
1000 вставок по возрастанию	Все ключи присутствуют, инварианты сохранены	Пройдено
1000 удалений по убыванию	Пустое дерево, корректный размер	Пройдено
25 000 случайных операций	Совпадение с <code>std::set</code>	Пройдено
Дубликаты и отсутствующие ключи	Возврат <code>false</code> без порчи дерева	Пройдено
Ввод с ошибкой / сохранение	Не менять исходное дерево	Пройдено
Слияние, копирование, перемещение	Корректные ключи и инварианты	Пройдено

Контрольная сборка и запуск `CTest` завершились сообщением «100% tests passed, 0 tests failed out of 1». Это подтверждает корректность реализованных тестовых сценариев, но не заменяет формального доказательства всех возможных состояний дерева.

Память для дерева оценивается как $O(n)$; дополнительно рекурсивные вставка и удаление используют $O(\log n)$ кадров стека. Накладные расходы вызваны векторами и указателями `unique_ptr` в каждой вершине. Это удобно для безопасного управления памятью, но менее компактно, чем реализация узлов на фиксированных массивах.

ЗАКЛЮЧЕНИЕ

В ходе курсовой работы была разработана программная модель 2–3-дерева на языке C++17. Реализованы построение из набора целых значений, поиск, вставка, удаление с балансировкой, объединение деревьев, определение следующего и предыдущего ключей и получение заданного количества следующих элементов. Перегружены операторы потокового ввода и вывода, позволяющие использовать единый текстовый формат при работе с консолью и файлами.

Полученное решение сохраняет основные инварианты многоузлового сбалансированного дерева поиска: каждый узел содержит один или два ключа, внутренний узел имеет два или три потомка, а все листья располагаются на одной глубине. Поиск, вставка, удаление и поиск соседних ключей имеют гарантированную трудоёмкость $O(\log n)$. Получение r следующих элементов занимает $O(\log n + r)$, что выгодно по сравнению с r независимыми поисками преемника.

Сравнение с альтернативами показывает, что 2–3-дерево является содержательным решением для упорядоченных динамических множеств, однако не универсально. Для обычных прикладных задач на C++ часто рациональнее применять стандартный `std::set`; для индексов внешней памяти — B-деревья или B+ деревья; для точного поиска без поддержания порядка — хеш-таблицы. Таким образом, преимущество представленной реализации состоит прежде всего в наглядной контролируемой балансировке и гарантированной высоте дерева.

Автоматизированная проверка с эталонным контейнером `std::set` и испытания потокового ввода и вывода завершились успешно. Использование `unique_ptr` обеспечивает владение узлами и автоматическое освобождение памяти, а метод `validate()` предоставляет дополнительный инструмент проверки корректности структуры после преобразований.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. Кнут Д. Искусство программирования. Том 3. Сортировка и поиск. — Москва: Вильямс, 2013. — 824 с.
2. konshyn. 2-3-дерево. Наивная реализация // Хабр. — 2016. — URL: <https://habr.com/ru/articles/303374/> (дата обращения: 08.10.2026).
3. Викиконспекты ИТМО. 2-3 дерево // Учебный электронный материал. — URL: https://neerc.ifmo.ru/wiki/index.php?title=2-3_дерево (дата обращения: 08.10.2026).
4. MaxRokatansky. Структура данных B-дерево / перевод материала Btech Smart Class // Хабр, блог OTUS. — 2019. — URL: <https://habr.com/ru/companies/otus/articles/459216/> (дата обращения: 08.10.2026).
5. Кормен Т. Х., Лейзерсон Ч. И., Ривест Р. Л., Штайн К. Алгоритмы: построение и анализ. 3-е изд. — Москва: Вильямс, 2013. — 1328 с.
6. imicah. Понимаем красно-черное дерево. Часть 1. Введение // Хабр. — 2021. — URL: <https://habr.com/ru/articles/555404/> (дата обращения: 08.10.2026).
7. Страуструп Б. Программирование. Принципы и практика с использованием C++. — Москва: Вильямс, 2016. — 1328 с.

Примечание. Учебный ресурс «Викиконспекты ИТМО» включён по перечню источников задания; персональный автор на странице не указан. В методических указаниях предпочтение отдано книгам и статьям с авторами.

ПРИЛОЖЕНИЕ А. СБОРКА И ПРИМЕРЫ ИСПОЛЬЗОВАНИЯ

В приложение вынесены команды сборки и короткие фрагменты использования открытого API. Полные исходные тексты TwoThreeTree.hpp, TwoThreeTree.cpp, main.cpp и test.cpp передаются отдельно вместе с проектом. Код программы размещён в отчёте текстом, а не изображениями.

```
g++ -std=c++17 -Wall -Wextra -Wpedantic -Iinclude \
    src/TwoThreeTree.cpp src/main.cpp -o demo
./demo

g++ -std=c++17 -O2 -Iinclude \
    src/TwoThreeTree.cpp tests/test.cpp -o tests_run
./tests_run
```

Листинг 10. Сборка консольного приложения и тестов (Unix-like OS)

```
TwoThreeTree tree({10, 20, 5, 15, 30, 25, 40});
tree.contains(15);           // true
tree.successor(15);          // 20
tree.predecessor(15);        // 10
tree.nextElements(12, 3);    // {15, 20, 25}
tree.insert(35);
tree.erase(20);
TwoThreeTree other({0, 15, 45, 35});
TwoThreeTree united = tree.merged(other);
```

Листинг 11. Пример обращений к основным методам класса TwoThreeTree

```
TwoThreeTree tree;
std::ifstream input("tree.txt");
input >> tree;
std::ofstream output("result.txt");
output << tree << '\n';
// Формат содержимого: N K1 K2 ... KN
```

Листинг 12. Пример чтения и сохранения дерева в файле

Пример исходного файла tree.txt: «7 10 20 5 15 30 25 40». После чтения и записи результирующий файл содержит: «7 5 10 15 20 25 30 40». Дубликаты вводимых значений игнорируются; при ошибочном формате чтение прекращается, оставляя ранее существовавшее дерево без изменений.